Smart Traffic Congestion Prediction

Live demo walkthrough and learning guide
Computer vision + machine learning

How to use this document

Part A: Live Demo Walkthrough. Follow this while you present. It lists exactly what to click and say in the running app, with timings and fixes if something breaks.

Part B: Learning Guide. Read this before presenting so you can explain every part: the data, the models, the computer vision, the fusion, and the newer features (lessons 11 to 15), plus a 2-minute script and interview questions.

Prepared 18 October 2026. The simulated challans are a demonstration, not real enforcement.

Contents

Part A: Live Demo Walkthrough 3

0. Before you present (do this 20 minutes early) . . . 3
1. Opening (30 seconds, on the header) . . . 3
2. Tab 1: Single camera (about 4 minutes) — must show . . . 3
3. Tab 2: Monitoring grid (1 minute) — must show . . . 4
4. Tab 3: Route (2 minutes) — must show . . . 4
5. Tab 4: Live e-Challan (3 minutes) — the showpiece . . . 4
6. Closing (1 minute): under the hood . . . 5
7. If something breaks . . . 5
8. Questions to expect . . . 5

Part B: Learning Guide 6

Lesson 1: Project Overview . . . 6
Lesson 2: System Architecture . . . 7
Lesson 3: Dataset and Feature Engineering . . . 9
Lesson 4: Machine Learning Models . . . 10
Lesson 5: Computer Vision . . . 11
Lesson 6: Live Prediction and Fusion Logic . . . 12
Lesson 7: Streamlit Dashboard . . . 13
Lesson 8: Advanced Features . . . 14
Lesson 9: Testing and Results . . . 15
Lesson 10: Limitations and Future Scope . . . 16
Lesson 11: Evaluating the Computer Vision . . . 17
Lesson 12: Tracking and Speed From Live Video . . . 18
Lesson 13: The Live e-Challan Tab . . . 19
Lesson 14: Time Zone, Speed Limits and Live Video . . . 21
Lesson 15: Route Map, Design Decisions and Bug Stories . . . 21
Complete 2-Minute Presentation Script . . . 22
Common Interview Questions and Short Answers . . . 23
Added Questions and Answers . . . 24
Study Order . . . 25

Part A: Live Demo Walkthrough

You are presenting by running the real app, not slides. Everything below uses the actual tab and button names. Total: about 12 minutes if you do everything, 6 if you only do the **must show** items.

0. Before you present (do this 20 minutes early)

1. **Pick the right time.** The live parts need daylight and traffic in California, so check the clock (California is 12.5 hours behind India until 1 Nov 2026, 13.5 hours after):

California time	India time (until 1 Nov)	Verdict
7-9 AM	7:30-9:30 PM	Best: commute traffic
4-6 PM	4:30-6:30 AM	Best: commute traffic
10 AM-3 PM	10:30 PM-3:30 AM	Fine
8 PM-5 AM	8:30 AM-5:30 PM	Poor: dark, almost no cars

   If your slot is in the poor window, the prediction, grid and route tabs still work; the live monitor will show "calibrating" because there are too few cars. Have a screen recording of it working ready.

2. **Run the readiness check:** `python demo_check.py`. It tests every online service and lists the busiest cameras with working video right now. Note the top one and use it in step 5.
3. **Start the app:** `.venv\Scripts\streamlit run app.py`, open http://localhost:8501.
4. **Warm everything up** by visiting every tab once and pressing Fetch live snapshot, Load grid and Start monitoring (then Stop). The first run of each loads models and is slow; the second is fast.
5. **Have backups:** screenshots of each tab, and a short screen recording of the Live e-Challan tab.

1. Opening (30 seconds, on the header)

Say: "This is a traffic congestion system. It uses **computer vision** to count vehicles on real live California highway cameras, and **machine learning** to predict what traffic should look like from the time of day and weather. It combines the two, and everything you'll see runs on real live data."

Point at the moving cars at the top: "They slow down when the latest reading is congested."

2. Tab 1: Single camera (about 4 minutes) — must show

Camera selection

- Click a dot on the map, or use the District / Filter / Camera boxes. Say: "478 real Caltrans cameras."
- Tick **Watch live video**. Say: "This is the actual live stream, played straight from Caltrans."

Computer vision

- Press **Fetch live snapshot**. Point at the boxes: "YOLOv8 detects each vehicle. I tuned its confidence on 162 frames I labelled myself, which cut the counting error from 7.4 to 4.6 vehicles."
- Point at the **vehicle counter** and **image density ratio**: "Density is how much of the picture the vehicles cover. That becomes the live congestion level."

Machine learning

- Open the **model radio** (LSTM / SVR / Linear Regression) and the **All three models** table. Say: "Three models trained on 48,000 hours of traffic data, with R-squared of 0.977, 0.96 and 0.948."
- Open **Why this prediction?** "Which inputs are unusual right now."

- Point at **Today's forecast**: "The shaded band is the model's typical error on held-out data, so the forecast isn't pretending to be exact."
- Open **When should I leave?** "Suggests a nearby hour with lower predicted congestion."

Fusion (the key idea)

- Point at the caption "Live camera reading: ... historical reading: ... final level takes whichever is more severe." Say: "A warning system should never say Low when the camera shows gridlock."
- If the **Unusual congestion detected** banner is visible, explain it: the live camera and the historical model disagree sharply, which usually means an accident or event.

Features to tick quickly

- **Use real weather here** (fills conditions from the camera's location).
- **Auto-refresh** (re-fetches on a timer and builds **Recent history**).
- **Download report (PNG)**.
- Mention the **Upload image** and **Snapshot URL** tabs work for any picture.

Optional (30 s): scroll to **Speed check**, press **Check speeds**, then **Identify vehicle color & type**.

3. Tab 2: Monitoring grid (1 minute) — must show

Press **Load grid** (6 cameras). Say: "Each camera gets its own live detection and badge." Point at a "camera unavailable" card: "About 1 in 5 Caltrans cameras serves a 'Temporarily Unavailable' image. My first version read that as an empty road. I found it by testing and fixed it."

4. Tab 3: Route (2 minutes) — must show

Choose **Search places**: type `Burbank Airport` and `Santa Monica Pier`, press **Find places**, pick the matches, press **Check route** (takes about 1 minute; talk while it runs). Say: "It finds the real road path, then every camera within 1.5 km of it, checks them all, and colors the road by each camera's live reading. The overall level is the worst segment." Show **Click the map** (drop two points) and **Pick cameras** in 15 seconds each.

5. Tab 4: Live e-Challan (3 minutes) — the showpiece

1. The camera box is pre-set to **I-5 at Olive** (or pick the camera `demo_check.py` recommended). Say: "Live video here, and the speed limit was filled in automatically from OpenStreetMap."
2. Press **Start monitoring**. Say what you see as it runs:
   - "The detector runs on the video about 4 times a second and tracks each car."
   - "For the first 25 seconds it learns the camera's geometry from the traffic itself. There is no calibration data for these cameras."
   - Then boxes show **estimated speeds**, and the badge and vehicle counter update live.
3. When a car is clearly over the limit a **simulated challan** appears with the car's picture, time, speed, limit and its source. (To force one in a quiet moment, lower the limit and margin first.)
4. Scroll down: **Generate demo challan** (made-up values), the **history** table, **Download history (CSV)**, and **Clear history**.
5. **Say this clearly:** "This is simulated. The speeds are estimates, not validated against radar, so I built in safeguards: it refuses to issue anything if the camera can't be calibrated, a car must stay over the limit for three measurements in a row, and a session issues at most 12 challans."

6. Closing (1 minute): under the hood

- "133 automated tests." (Run `python -m pytest -q` in a terminal if you want to show it.)
- Open `cv_eval/results/report.md`: "The evaluation behind the numbers, including the ideas I tried and rejected: bigger models, sliced detection, a learned classifier."
- Open the README **Limitations and future work**. Say: "The historical model was trained on Minneapolis data, speeds are unvalidated, and the labels are approximate. I'd rather state that than hide it."

7. If something breaks

Problem	What to do
Internet or Caltrans down	Show your screenshots and recording; the README and `cv_eval` results work offline
One camera says "unavailable"	Pick another; it happens to about 1 in 5
Live monitor says "speeds not reliable"	Say "that is the safeguard working", then switch to the recommended camera
Route check is slow	Keep talking; it checks 16 cameras (about 1 minute)
Speed limit says "assumed 65 mph"	OpenStreetMap was slow; it updates by itself when it answers
Something looks frozen	Reload the page and continue

8. Questions to expect

- **Why take the more severe of the two readings?** To avoid false reassurance.
- **What did you train yourself?** The three prediction models. The detector is pretrained; I tuned and evaluated it.
- **How accurate is the speed?** Honestly, rough: it is calibrated from the traffic itself, not radar.
- **Is the challan real?** No: it is simulated and labelled as such on every card.
- **Why is the historical model trained on Minneapolis data?** It was the best public hourly dataset available; the fix is California sensor data (PeMS), listed as future work.
- **Did you use AI to help?** Answer honestly per your college's rules, and be ready to explain any part.

Part B: Learning Guide

This guide explains the full project in a presentation-friendly way. Use it to learn the project flow, understand the technology stack, and prepare interview answers.

Lesson 1: Project Overview

What the project does

This project predicts road traffic congestion by combining two signals:

1. Historical machine learning prediction
2. Live camera-based computer vision detection

The historical ML side predicts expected traffic volume using time, weather, holiday, and recent traffic-volume features. The computer vision side analyzes a live traffic camera image using YOLO to detect vehicles and estimate how crowded the road looks.

The final result is shown as:

```
Low / Moderate / High / Severe
```

One-line explanation

> *My project predicts traffic congestion by combining historical ML models with live camera-based vehicle detection, and displays the result in a Streamlit dashboard.*

Why this project is useful

Traditional traffic prediction often depends only on historical data. But real roads can suddenly change because of accidents, weather, events, or roadblocks. This project adds a live camera signal so the prediction can react to real-time conditions.

Main project flow

```
Historical traffic data
        |
        v
Feature engineering
        |
        v
Train ML models
        |
        v
Predict expected traffic volume


Live traffic camera image
        |
        v
YOLO vehicle detection
        |
        v
Estimate live congestion


ML result + CV result
        |
        v
Fusion logic
        |
        v
Final congestion prediction
        |
        v
Streamlit dashboard
```

Presentation paragraph to memorize

> *This project predicts traffic congestion using two independent signals. First, historical traffic data is used to train ML models that predict traffic volume based on time, weather, and lag features. Second, a live camera image is analyzed using YOLO to detect vehicles and estimate image density. These two outputs are fused by choosing the more severe congestion level, because in a warning system under-reporting visible congestion is worse than over-reporting it. The final result is displayed in a Streamlit dashboard.*

Lesson 2: System Architecture

Four main layers

```
1. Data Layer
   Historical CSV, live camera feed, weather API

2. Model Layer
   Linear Regression, SVR, LSTM, YOLO

3. Logic Layer
   Feature engineering, live prediction, fusion, anomaly detection

4. UI Layer
   Streamlit dashboard
```

Main files and responsibilities

File	Role
`src/data_loader.py`	Loads CSV and creates model features
`src/train_regression.py`	Trains Linear Regression and SVR
`src/train_dl.py`	Trains LSTM
`src/evaluate.py`	Trains all three models and compares them
`src/cv_module.py`	Runs YOLO vehicle detection
`src/live_predict.py`	Builds live features, predicts volume, and fuses labels
`src/live_cameras.py`	Fetches live Caltrans camera data
`src/weather.py`	Fetches current weather from Open-Meteo
`src/speed_estimation.py`	Estimates vehicle speed from live video
`src/anpr.py`	Attempts license plate OCR
`src/vehicle_attributes.py`	Estimates vehicle color and body type
`src/echallan.py`	Generates simulated challan summaries
`src/prediction_log.py`	Stores local prediction history
`src/report.py`	Builds downloadable PNG reports
`src/camera_time.py`	California time for model inputs (the cameras' clock)
`src/speed_limits.py`	Posted speed limit for a camera's road from OpenStreetMap
`src/vehicle_tracking.py`	Vehicle tracking, camera calibration and speed estimation
`src/live_monitor.py`	Background live monitor: detection, speed, automatic challans
`src/live_video.py`	Plays a camera's live video stream in the page
`src/route_map.py`	Road path, place search, cameras along a route
`src/ui_theme.py`	The control-room look: header, badges, counter
`cv_eval/`	Labelled frames and the vision evaluation (Lesson 11)
`app.py`	Streamlit dashboard connecting everything

Runtime flow

When the user selects a live camera:

1. `app.py` fetches a live camera frame.
2. `cv_module.py` detects vehicles using YOLO.
3. `live_predict.py` builds a feature row using current time, weather, and lag estimates.
4. Saved ML models predict traffic volume.
5. Predicted volume is converted into a congestion label.
6. CV density is converted into a congestion label.
7. The two labels are fused by taking the more severe one.
8. The final result appears in the Streamlit dashboard.

Interview answer

> *The project is modular. The data pipeline is handled in `data_loader.py`, model training is split between `train_regression.py` for Linear Regression and SVR and `train_dl.py` for LSTM, computer vision is handled in `cv_module.py`, and live inference plus fusion is handled in `live_predict.py`. Finally, `app.py` connects everything into a Streamlit dashboard.*

Lesson 3: Dataset and Feature Engineering

Dataset

The project uses the Metro Interstate Traffic Volume dataset. It contains hourly traffic-volume readings along with weather and time-related information.

The target variable is:

```
traffic_volume
```

The model tries to predict this value.

Important features

The project creates features such as:

Feature Type	Examples	Why it matters
Time features	hour, day of week, month	Traffic depends strongly on time
Cyclical features	`hour_sin`, `hour_cos`	Helps model understand that 23:00 and 00:00 are close
Weather features	temperature, rain, snow, clouds	Weather can affect traffic
Holiday feature	`is_holiday`	Holidays change travel behavior
Lag features	`lag_1h`, `lag_3h`, `lag_24h`	Recent traffic is very useful for prediction
Rolling feature	`rolling_mean_3h`	Smooths recent traffic trend

Why sin/cos time features are used

Raw hour values can mislead a model. For example, hour 23 and hour 0 are numerically far apart, but in real time they are next to each other.

Sin/cos encoding fixes this by representing time as a cycle.

Why lag features matter

Traffic is time-dependent. If traffic was high one hour ago, it is likely still high now. Lag features give the model recent history.

This is one of the biggest reasons the Linear Regression and SVR models became accurate.

Chronological split

The project uses chronological train/validation/test splitting instead of random splitting.

This matters because traffic data is time-series data. If future rows are mixed into training, the model can accidentally learn from the future. That is called data leakage.

Interview answer

> *I used cyclical time features because time repeats daily and weekly. I also added lag and rolling features because traffic volume is strongly dependent on recent traffic. Since this is time-series data, I used chronological splitting instead of random splitting to avoid data leakage from future records.*

Lesson 4: Machine Learning Models

The project trains three models:

```
Linear Regression
SVR
LSTM
```

Linear Regression

Linear Regression is used as a baseline model. It is simple, fast, and explainable.

It became much stronger after adding:

1. Sin/cos cyclical time features
2. Lag and rolling traffic features

SVR

SVR means Support Vector Regression. The project uses an RBF kernel, which can model nonlinear relationships better than simple Linear Regression.

SVR is slower to train on large datasets, so it is trained on an 8,000-row subsample for speed.

Hyperparameters were tuned separately using `RandomizedSearchCV` with `TimeSeriesSplit`.

LSTM

LSTM is a deep learning model used for sequence/time-series prediction.

In this project, it looks at a 24-hour window of previous features to predict traffic volume.

The LSTM performs best, but the gain over SVR is not huge because the lag features already give the classical models strong time-history information.

Metrics

The main evaluation metrics are:

Metric	Meaning
MAE	Average absolute error
RMSE	Penalizes larger errors more strongly
R2	How much variance the model explains

Metrics are calculated in:

```
src/train_regression.py
src/train_dl.py
```

`src/evaluate.py` trains all three models, prints the comparison, and creates the comparison plot using Matplotlib.

Interview answer

> *I trained Linear Regression as a baseline, SVR with an RBF kernel for better nonlinear modeling, and an LSTM for time-series sequence learning. The LSTM gave the best result, but SVR was also strong because lag features already captured recent traffic behavior.*

Lesson 5: Computer Vision

Purpose

The computer vision module gives the project a live real-world signal.

Instead of relying only on historical data, the project looks at a current traffic camera frame and detects vehicles.

YOLO

YOLO is used for object detection. It detects vehicle classes such as:

```
car
motorcycle
bus
truck
```

The project uses YOLOv8s with a detection confidence of 0.08, tuned on labelled frames (Lesson 11).

OpenCV

OpenCV is used for:

1. Loading images
2. Reading video frames
3. Handling image arrays
4. Drawing annotations

Density calculation

After YOLO detects vehicles, the project calculates how much of the image is covered by vehicle bounding boxes.

```
density_ratio = total vehicle bounding-box area / frame area
```

Then density is mapped to:

```
Low / Moderate / High / Severe
```

Limitation

YOLO can struggle on low-resolution, nighttime, or blurry traffic camera images. This is a real-world limitation and should be mentioned honestly.

Interview answer

The CV module uses YOLOv8s to detect vehicles in a traffic camera frame. OpenCV loads the image, YOLO finds vehicle bounding boxes, and the system calculates an image-density ratio from the total vehicle area. That density is mapped to a congestion label.

Lesson 6: Live Prediction and Fusion Logic

Why fusion is needed

The historical ML model and live camera analysis give different kinds of information.

Historical model:

```
What traffic is expected to be like at this time
```

Live camera:

```
What the road looks like right now
```

Both are useful, so the project combines them.

Fusion strategy

The final label is the more severe of the two labels:

```
final_label = max_severity(historical_label, cv_label)
```

Example:

Historical ML	Live CV	Final
Low	Severe	Severe
High	Moderate	High
Moderate	Moderate	Moderate

Why not weighted average?

A weighted blend was tried first. But it failed in an important case: a visibly gridlocked road was detected as Severe by CV, but the historical model predicted Low, and the weighted blend still reported Low.

That is dangerous for a warning system.

So the project chooses the more severe signal to avoid under-reporting visible congestion.

Anomaly detection

If the historical model and live camera disagree by two or more severity levels, the project flags it as unusual congestion.

This can indicate:

1. Accident
2. Road closure
3. Event traffic
4. Sudden congestion not present in historical data

Interview answer

> *The fusion logic takes the more severe of the historical ML label and the live CV label. I chose this because a previous weighted blend under-reported a visibly jammed image as Low. For a warning system, missing real congestion is worse than being slightly conservative.*

Lesson 7: Streamlit Dashboard

Purpose

The Streamlit app is the user interface. It lets a user:

1. Pick a live Caltrans camera
2. Upload an image
3. Paste a snapshot URL
4. Set weather conditions
5. Choose a model
6. View final congestion prediction
7. See detected vehicles
8. View forecast chart
9. Download report
10. Check route or multiple cameras

Important Streamlit concepts

Concept	How it appears
Widgets	sliders, buttons, select boxes, checkboxes
Tabs	Single camera, Monitoring grid, Route, Live e-Challan
Session state	remembers selected camera, current frame, speed results
Reruns	Streamlit reruns the script after user interaction
Fragments	used for auto-refresh without blocking the whole app
Cache	avoids rerunning expensive detection/model prediction unnecessarily

Important bug story

Streamlit tabs still execute their code even when the tab is not visually active. This caused stale inputs from another tab to overwrite the current camera frame.

The project fixes this by tracking which source acted most recently:

```
frame_source = live / upload / url
```

Auto-refresh bug story

An early auto-refresh design used `sleep` and rerun. That blocked the app and made the UI unstable.

The project now uses Streamlit fragments and checks whether enough time has passed before fetching again.

Interview answer

> *Streamlit reruns the script whenever a widget changes, so I used session state to preserve selected cameras, frames, speed results, and source tracking. I also used caching for expensive YOLO/model calls and fragments for safe auto-refresh.*

Lesson 8: Advanced Features

Live cameras

`src/live_cameras.py` fetches real public Caltrans CCTV camera data.

The dashboard lets the user select district, route/location, and camera.

Real weather

`src/weather.py` fetches current weather from Open-Meteo using the camera's latitude and longitude.

This can auto-fill:

```
temperature
rain
snow
cloud cover
weather type
```

Forecast chart

The app predicts traffic for every hour of the day and shows it with Altair.

It also displays an error band using each model's typical MAE, so the forecast does not look falsely exact.

Pydeck map

The app uses pydeck for the camera map.

Important bug story:

Per-row accessors for color/radius broke click selection on the map. The fix was to separate the map into layers:

1. Non-pickable congestion-color layer
2. Pickable camera layer
3. Selected-camera highlight layer

Route tab

The route tab lets users pick cameras, search places, or click the map. It draws the real road path colored by nearby cameras (Lesson 15). The final route congestion is the worst segment.

This makes sense because one jammed segment can affect the whole route.

Speed estimation

`src/speed_estimation.py` estimates vehicle speeds from live HLS video streams.

It uses:

1. YOLO detections across multiple frames

2. Nearest-centroid tracking
3. Stream timestamps
4. Lane-width calibration

Important detail:

It uses the stream's internal timestamp, not wall-clock time, because HLS video can download buffered frames faster than real time.

This is the older speed check in the Single camera tab. The newer Live e-Challan tab fits a camera model from the traffic itself (Lesson 12).

ANPR / OCR

`src/anpr.py` uses EasyOCR to attempt license plate recognition from a selected vehicle crop.

Limitation:

Real traffic camera footage often has blurry, angled, hidden, or unreadable plates.

Vehicle attributes

`src/vehicle_attributes.py` estimates:

1. Vehicle color using image color clustering
2. Vehicle body type using CLIP zero-shot classification

This is best-effort and not guaranteed.

Simulated e-challan

`src/echallan.py` generates an illustrative fine summary for a speeding vehicle.

It is a demo only:

```
No real fine is issued.
No government system is connected.
No payment is charged.
```

JSON logs

The project stores local logs for:

1. Prediction history
2. Simulated challan history

These are local JSON files and are not sent anywhere.

Lesson 9: Testing and Results

Test suite

The project has pytest tests covering:

1. Data loading and feature engineering
2. Fusion logic
3. Live prediction
4. Weather mapping
5. Report generation

6. Prediction logs
7. Speed estimation math
8. OCR behavior
9. E-challan logic
10. Vehicle attributes

The test suite passed:

```
133 passed
```

Why tests matter

Tests prove that the important logic works and that future changes do not break existing behavior.

Strong presentation point

> *I added tests not only for the basic model pipeline, but also for edge cases like fusion behavior, report generation, weather mapping, OCR fallback, speed estimation math, and local logs.*

Interview answer

> *I used pytest to test the core behavior of the project. The tests cover data processing, live prediction, fusion logic, reports, weather, speed estimation, OCR, challan generation, and vehicle attributes. This helps make the project more reliable and easier to change.*

Lesson 10: Limitations and Future Scope

Main limitations

1. The historical model is trained on Minneapolis-area data but the live cameras are in California.
2. YOLO can struggle with low-resolution or nighttime images.
3. Speed estimates are not validated against radar (Lesson 12); the older speed check needs manual lane-width calibration.
4. OCR is often unreliable on real traffic camera footage.
5. The system is a demonstration project, not a government-certified traffic enforcement system.
6. The fusion rule is conservative, but not learned from paired ground-truth camera-volume data.

Why the dataset mismatch matters

The historical model learns traffic patterns from one dataset, while live cameras come from Caltrans. Without paired camera + sensor data for the exact same locations, the model cannot be perfectly calibrated to each camera.

Future improvements

1. Train on paired camera and sensor data from the same road locations.
2. Fine-tune YOLO on traffic-camera images, especially nighttime/low-resolution footage.
3. Add a stronger multi-object tracker for speed estimation.
4. Add camera calibration for more accurate speed measurement.
5. Improve OCR with better plate-specific preprocessing.

6. Deploy the app publicly with a reliable backend.
7. Add live incident/event feeds.

Interview answer

> *The main limitation is that the historical dataset and live camera feeds are not perfectly paired by location, so the model is best understood as a demonstration of an end-to-end architecture. Accuracy could improve with paired traffic sensor and camera data, YOLO fine-tuning on local traffic cameras, and proper camera calibration for speed estimation.*

Lesson 11: Evaluating the Computer Vision

Why this lesson exists

The R-squared of 0.977 belongs to the historical model only. For a long time the computer vision part had no accuracy number at all. This lesson is the measurement of the vision part, found in `cv_eval/`.

The evaluation set

Item	Value
Frames	162 real Caltrans frames, all 12 districts, captured 2026-10-13 around 09:00-09:25 PDT
Extra frames	Interstate and US-route cameras from the busy districts, so there are congested examples
Labels per frame	vehicle count, congestion class (Low / Moderate / High / Severe), usable yes/no
Usable	124 frames (Low 92, Moderate 22, High 8, Severe 2)
Unusable	33 "Temporarily Unavailable" images, 3 glare, 1 haze, 1 parking lot

Honest note about the labels: they were made by looking at the frames (with Claude's help), not taken from a sensor. Counts are approximate (small distant cars are counted, which makes the detector look worse) and the classes are a visual judgement. Only 10 frames are High or Severe, so results for those classes are weak evidence. Re-check them with `python cv_eval/make_sheets.py --labels`.

Finding 1: one in five cameras was "Temporarily Unavailable"

33 of 162 sampled cameras served a mostly white placeholder image. A detector finds no cars in it, so the app reported an empty road ("Low"). The placeholder is about 86 percent near-white pixels, and the most washed-out real frame was 22 percent, so a 60 percent cutoff separates them. `analyze_frame` now raises `CameraUnavailableError`, and the UI shows "camera unavailable" instead of a badge.

Finding 2: the detector undercounted, and a lower confidence fixed most of it

Setting (YOLOv8s, 640 px)	Count error (MAE)	Average miss	Within tolerance
Confidence 0.25 (YOLO default)	7.43 vehicles	-7.3 (too low)	39 percent
Confidence 0.08 (shipped)	4.55 vehicles	-2.5	59 percent

On dense frames (21 or more vehicles) the error fell from 23.5 to 12.6; on near-empty frames it moved from 1.55 to 1.41, so empty roads did not get worse. The app now uses `DEFAULT_CONF = 0.08`.

Finding 3: what did not help

- **A bigger model:** YOLOv8m at 0.08 reached 4.15 (a small gain) but takes 0.56 seconds per frame against 0.24, which hurts when a route checks 16 cameras. The app stays on YOLOv8s.
- **Higher input size (1280 px):** no gain.
- **Refitting the density thresholds** (0.05 / 0.15 / 0.30) on labelled data with cross-validation: worse macro-F1 than the original thresholds once the detector was fixed.
- **Sliced inference** (also searching four overlapping windows so far cars have more pixels): 4.55 to 4.32, only about 5 percent, at roughly five times the compute. Kept as an option, not the default.
- **A learned congestion classifier** (logistic regression, random forest, gradient boosting): balanced accuracy 0.41-0.54 against 0.62 for the plain thresholds. 124 frames is too few for a learned model to win.

Reading the label metrics correctly

74 percent of usable frames are Low, so "always answer Low" already scores 0.74 accuracy. That is why the report also gives **balanced accuracy** (average recall per class) and **macro-F1**.

Setting	Accuracy	Balanced accuracy	Macro-F1
Before tuning (confidence 0.25)	0.73	0.32	0.33
Shipped (confidence 0.08)	0.75	0.62	0.64
Always answer "Low"	0.74	0.25	-

Caveats to say out loud

The confidence value was picked from a handful of options on the same frames it is scored on, so the gain is slightly optimistic. The set is daytime only, from one 25-minute window. Dense traffic is still undercounted (one wide freeway frame with about 45 vehicles was read as 9). Nothing was fine-tuned: that needs bounding boxes, not the counts collected here.

Interview answer

> *I measured the vision part on 162 live frames I labelled across all 12 districts. That found two real problems: a fifth of the cameras serve an unavailable image that looked like an empty road, and the detector at its default confidence undercounted by about 7 vehicles. Lowering the confidence cut the counting error from 7.4 to 4.6. I also tried bigger models, sliced inference, and a learned classifier, and kept the simple version because they did not clearly help. The labels are approximate and daytime only, so I treat the numbers as evidence, not proof.*

Lesson 12: Tracking and Speed From Live Video

The problem

Speed is distance over time. The time is easy (the video stream has a clock). The distance is hard: the cameras publish no calibration, so a pixel on the screen has no known size in metres.

The pipeline (src/live_monitor.py and src/vehicle_tracking.py)

1. **Frames:** the stream is 15 frames per second. Every 4th frame is analysed (about 3.75 per second) and its time is read from the stream's own clock, because network delays make wall-clock time wrong.
2. **Detect and track:** YOLO finds the vehicles. A simple tracker predicts where each vehicle should be next from its recent motion and matches the nearest detection, so the same car keeps one id.

3. **Fit a camera model from the cars themselves** (first about 25 seconds). For a camera looking along a flat road, a car's bottom edge in the image (y) and its pixel width (w) follow a line: `y = horizon + a * w`. Fitting that line over many cars gives the horizon row and the camera height (`a * 1.8 m`, taking a typical car to be 1.8 m wide). If the fit is not convincing (too few cars, no trend, implausible height), speeds are switched off for that camera.
4. **Pixels to metres:** the focal length is assumed from a typical 62 degree field of view. Then `distance ahead = f * height / (y - horizon)` and `sideways = (x - centre) * distance / f`.
5. **Speed:** for each car, take its last 2.5 seconds, fit a straight line to its ground position over time, and take the slope. Only cars between 8 and 75 metres from the camera count.
6. **One correction:** the zoom was only assumed, so speeds are scaled so the median tracked car matches the "typical traffic speed" setting (90 percent of the posted limit by default). If that needs a correction outside 0.4 to 2.5 times, the camera is marked unreliable and issues nothing.
7. **Flag:** the car must stay over the limit plus a margin for 3 measurements in a row.

The older method (Single camera, Speed check)

You measure a lane's width in pixels and type it in. Speed is pixels moved times (3.7 m divided by lane pixels) divided by time. It ignores perspective and depends on your measurement.

How far to trust it

- **Verified:** a synthetic car moving at a known speed reads within 4 percent.
- **Not verified:** there is no radar or other ground truth. On the real cameras tried, the raw model needed a correction of about 2 times, and far-away cars read much faster than near ones in the same traffic. That is why the distance band and the traffic-speed scaling exist.
- **What the numbers mean:** treat speeds as "faster or slower than the rest of the traffic", not exact mph.

Interview answer

> *There is no calibration data for these cameras, so I fit a flat-road camera model from the traffic itself: a car's ground row and width follow a line that gives the horizon and camera height. With an assumed focal length I convert image positions to road distance, track each car over a couple of seconds, and take the slope as speed. Because the zoom is unknown I scale speeds to a typical traffic speed, and if the model can't be calibrated the app refuses to issue anything. My math tests pass on synthetic cars within 4 percent, but I haven't validated against radar, so I describe the speeds as estimates.*

Lesson 13: The Live e-Challan Tab

What you see

One tab with, top to bottom: camera selection (district, filter, camera), the camera's live video, the automatic speed limit, a Start monitoring button, the annotated live detection (boxes with estimated speeds, a vehicle counter, a congestion badge), the challans issued for real cars, then a form to make a challan from made-up values, the challan history, a CSV download and a clear-history button.

How it works

- **Live video:** an hls.js player runs in your browser and plays Caltrans's stream directly (their server allows this). Nothing is proxied or recorded by the app.
- **The monitor:** a background thread (`LiveMonitor`) reads the stream and runs detection, tracking and speed. It uses its **own** YOLO instance because the app's shared model is not safe to use from two threads. The page polls it once a second with a Streamlit fragment, and the monitor stops itself after the run time or if the page stops asking (45 seconds).
- **A challan:** when a car is clearly over the limit, the monitor crops its picture, optionally reads the plate and identifies its color and type, and builds a simulated challan. Plate reading and identification run in a separate worker so the video does not stall.
- **What a challan shows:** reference number, time in California time with its UTC offset, location, vehicle (with a note if no plate could be read), appearance, estimated speed, speed limit and where it came from, the illustrative fine in rupees, and the simulation disclaimer.
- **Privacy:** evidence pictures stay in memory only. The on-disk history holds text fields only.

Safeguards

3 confirmations in a row; at most 12 challans per session, at least 3 seconds apart; no challans at all if the camera could not be calibrated; a warning if the limit is set below typical traffic speed (which would flag most cars).

Plate, color, make and model

- **Plate (EasyOCR):** often unreadable on real footage; the challan then says "NOT READ".
- **Color and body type:** CLIP zero-shot and color clustering; best-effort guesses.
- **Make and model:** tried and gated. The median detected car is only 19 pixels wide, and even the 20 largest crops gave 5-15 percent confidence, so crops under 70 px are refused and a model is reported only above 30 percent confidence. On these cameras it will rarely say anything, on purpose.

Bug stories (good for interviews)

- **Hidden feature:** the e-challan used to appear only after a speed check found a speeder, so nobody saw it. It now has its own tab.
- **Runaway challans:** a test with the limit below normal traffic speed produced 293 challans in two minutes. That led to the cap, the spacing, and the warning.
- **Wrong wording:** a challan with no plate said "entered manually". It now says no plate could be read.

Interview answer

> *The Live e-Challan tab watches a real camera, tracks vehicles, and generates a clearly-labelled simulated challan from a real observed vehicle, with its picture, estimated speed, time, location and the posted limit. It runs in a background thread with its own detector, has safeguards against false or runaway challans, and refuses to issue anything when the camera can't be calibrated. It is a demonstration of the pipeline, not enforcement.*

Lesson 14: Time Zone, Speed Limits and Live Video

The time-zone bug

The historical models take an hour, weekday and month. The app used the computer's own clock. A laptop on India time (16:17) asked the models about Sunday afternoon rush hour while it was 03:46 in California, and the app showed a false "Unusual congestion detected". Found by a full test pass. Fix: `src/camera_time.py` gives California time (`America/Los_Angeles`), used for predictions, the forecast, the report footer and challan timestamps.

The automatic speed limit (src/speed_limits.py)

OpenStreetMap stores a `maxspeed` tag on many roads (Overpass API, free, no key). The app finds the road the camera watches and reads it:

1. A nearby road whose route reference matches the camera's (so a camera on an overpass reports the freeway, not the street crossing it), nearest first.
2. Otherwise the nearest motorway, then the nearest road of any kind.
3. If the road has no tag, a typical value for its class is used and labelled "assumed".

Checked live: the I-110 Arroyo Seco Parkway reports 55 mph and the I-5 freeway 65 mph, matching the posted limits.

Why it never blocks the page: the public Overpass servers are often slow or busy. The lookup runs in a background thread, the page shows a clearly-labelled provisional value (65 mph for Interstate and US routes) and updates itself when the real answer arrives. It tries three mirrors, and a found limit is cached on disk so each camera needs only one successful lookup. Limits are only as good as what OpenStreetMap contributors entered.

Live video in the browser

An hls.js player (version pinned) plays the camera's HLS playlist. About two thirds of cameras have a stream, and many of those are offline at any moment, so a dead stream shows a message instead of a blank box.

Interview answer

> *I get the posted speed limit from OpenStreetMap by finding the road the camera is on and reading its maxspeed tag. Because the public server is slow, I do it in a background thread with a provisional value, mirrors, retries and a disk cache. I also found and fixed a bug where the models used the laptop's clock instead of California's.*

Lesson 15: Route Map, Design Decisions and Bug Stories

The Route tab

Three ways to build a route, all giving the same result:

Mode	How
Pick cameras	choose 2 or more cameras in a district
Search places	type a start and destination (Nominatim place search, one request per second)
Click the map	click once for the start, again for the destination

The app gets the real road path from OSRM (OpenStreetMap routing, free), finds every camera within 1.5 km of it (up to 16, evenly spaced), checks each one, and colors the road by the nearest camera: solid within 2 km, faded 2-8 km away (a weaker guess), gray beyond 8 km. The overall route level is the worst segment. Colors are estimates from scattered snapshots, not continuous traffic data like a navigation app.

Design decisions worth mentioning

- **Per-segment live reading:** each route segment and each grid badge shows that camera's own live reading. The historical label is one value for the whole time and weather, so putting it into every segment put a "Moderate" floor under all of them. It is applied once, to the overall reading.
- **Free services only:** Caltrans, OSRM, Nominatim, Open-Meteo and OpenStreetMap need no API key.
- **A dark control-room theme** with moving cars that slow down when traffic is congested, glowing badges, and an animated counter, all in CSS.

Bug stories (a good list for questions)

Bug	What happened	Fix
Stale selectbox label	Changing a filter crashed with a TypeError	validate the stored value and reset
Auto-refresh loop	A Streamlit fragment also runs on every full rerun, causing an endless refresh	elapsed-time guard
Everything "Moderate"	the shared historical label was fused into every camera	per-camera live label, fuse once
Placeholder as empty road	"Temporarily Unavailable" images read as Low	detect and refuse them
Wrong clock	models used the laptop's time zone	California time
Runaway challans	293 in one test	cap, spacing, warning
Cropped map	a tall route was cut off in a short wide map	a proper fit-to-view calculation

What is distinctive (for "what makes it unique")

Not any single piece, but the combination: a historical model and live computer vision fused with a stated reason; real, messy live data with real failures found and fixed; measuring the vision part and keeping only what helped; and going beyond a dashboard to routes, tracking, speed, and a simulated enforcement pipeline, all with the limits written down.

Complete 2-Minute Presentation Script

My project is called Smart Traffic Congestion Prediction. The goal is to estimate road congestion by combining historical machine learning with live computer vision.

The historical side uses the Metro Interstate Traffic Volume dataset. I first clean the data and create features such as cyclical time features, weather features, holiday information, and lag features like traffic volume from one, three, and twenty-four hours ago. These features are used to train three models: Linear Regression, SVR, and LSTM.

Linear Regression acts as a simple baseline, SVR captures nonlinear patterns, and LSTM handles time-series sequences using a 24-hour window. The models are evaluated using MAE, RMSE, and R2.

The live side of the project uses YOLOv8s to detect vehicles from a traffic camera frame. It counts cars, buses, trucks, and motorcycles, then estimates image density based on vehicle bounding-box area. This gives a live camera-based congestion label.

The final prediction combines the historical ML label and live CV label using a conservative fusion rule: it chooses the more severe congestion level. I used this because a weighted blend can under-report visible congestion, which is risky for a warning system.

The full system is displayed in a Streamlit dashboard. Users can select live Caltrans cameras, upload images, use real weather, view forecasts, check routes, download reports, and run experimental features like speed estimation, OCR, and simulated challan generation.

Beyond the prediction, a Live e-Challan tab runs detection and tracking on the live video, estimates speeds, and generates clearly labelled simulated challans from real observed cars. I measured the vision part on 162 labelled frames, which cut the counting error from 7.4 to 4.6 vehicles, and I documented what did not work.

The project is modular, with separate files for data loading, model training, computer vision, live prediction, weather, reports, and UI. I also added a pytest suite covering the core logic and edge cases. The main limitation is that the historical dataset and live camera locations are not perfectly paired, so future work would involve training on paired camera-sensor data and fine-tuning YOLO for traffic-camera footage.

Common Interview Questions and Short Answers

Q1. What problem does your project solve?

It predicts road congestion using both historical traffic patterns and live camera conditions.

Q2. Why did you use both ML and computer vision?

Historical ML predicts expected traffic, while computer vision captures what is happening right now. Combining them makes the system more responsive.

Q3. Why use lag features?

Traffic is strongly dependent on recent traffic. Lag features give the model memory of recent conditions.

Q4. Why use sin/cos features?

Time is cyclical. Sin/cos encoding helps the model understand that hour 23 and hour 0 are close.

Q5. Why chronological split?

Because this is time-series data. Random splitting could leak future information into training.

Q6. Why use three models?

Linear Regression gives a baseline, SVR captures nonlinear patterns, and LSTM models sequences.

Q7. Why is the LSTM improvement over SVR not huge?

Because lag and rolling features already give SVR strong recent-history information.

Q8. How does YOLO help?

YOLO detects vehicles in live camera frames, allowing the system to estimate real-time congestion visually.

Q9. Why take the more severe fusion label?

Because under-reporting visible congestion is worse than being conservative in a warning system.

Q10. What are the main limitations?

Dataset-camera mismatch, low-resolution camera images, manual speed calibration, and unreliable OCR on real footage.

Q11. What would you improve next?

Use paired camera-sensor data, fine-tune YOLO on traffic footage, add better tracking, and deploy the app.

Added Questions and Answers

Q12. How is speed calculated?

By tracking each car across frames and converting image position to road distance with a camera model fitted from the traffic itself, then scaling to a typical traffic speed. It is an estimate, not a measurement.

Q13. How accurate is the speed?

Honestly, rough. The math is tested on synthetic cars (within 4 percent) but it is not validated against radar, so I describe it as "faster or slower than the rest of the traffic".

Q14. Is the challan real?

No. Every challan is simulated and labelled as such. Nothing connects to any authority or registry.

Q15. Why is the historical model trained on Minneapolis data?

It was the best public hourly dataset with weather and traffic volume that I could use. It does not match California, so its output is a time-of-day and weather baseline. The fix is California sensor data (PeMS).

Q16. How did you evaluate the vision part?

I labelled 162 live frames from all 12 districts and measured counting error and label accuracy. Tuning the confidence cut the error from 7.4 to 4.6 vehicles. I also tried bigger models, sliced inference and a learned classifier and kept the simple version.

Q17. What did you train yourself?

The three prediction models (Linear Regression, SVR, LSTM). The detector is pretrained; I tuned and evaluated it.

Q18. Can it identify the car's make and model?

Mostly no. The median car on these cameras is 19 pixels wide, so the app only reports a make or model when the crop is big enough and the model is confident, which is rare.

Q19. How does the app get the speed limit?

From OpenStreetMap, by finding the road the camera watches. It runs in the background with a fallback if the service is slow.

Q20. What are the biggest limitations?

The historical model's data mismatch, unvalidated speeds, approximate labels from a small daytime set, and cameras that are offline or have a sideways view.

Study Order

Use this order while learning:

1. `README.md`
2. `src/data_loader.py`
3. `src/train_regression.py`
4. `src/train_dl.py`
5. `src/evaluate.py`
6. `src/cv_module.py`
7. `src/live_predict.py`
8. `src/live_cameras.py`
9. `src/weather.py`
10. `src/speed_estimation.py`
11. `src/anpr.py`
12. `src/vehicle_attributes.py`
13. `src/echallan.py`
14. `src/prediction_log.py`
15. `src/report.py`
16. `app.py`
17. `tests/`
18. `src/vehicle_tracking.py`
19. `src/live_monitor.py`
20. `src/speed_limits.py`
21. `src/camera_time.py`
22. `src/route_map.py`
23. `cv_eval/evaluate_cv.py`